PART 4: SOFTWARE COMPONENTS

Section 1: System Requirements & Constraints

1. Core User Workflows

Customer discovery across buy-side and sell-side participants revealed three primary workflows the system must support.

Comps Identification

The most universally painful step in the current process. Buy-side traders and portfolio managers manually search for comparable CLO tranches by filtering on attributes like tenor, MVOC, manager tier, and coupon — a process that is time-consuming, inconsistent, and unscalable when pricing multiple bonds in a single session. The system must automate this step through an embedding-based similarity search that returns ranked, explainable comps.

Pricing / OAS Estimation

Once comps are identified, practitioners generate a pricing view — currently through informal triangulation in Excel. The system must produce a quantitative pricing estimate (OAS and/or price) supported by a transparent, interpretable model. Practitioners at Ellington and Barclays both indicated that explainability and precision are prerequisites for adoption; a black-box output will not clear internal model validation hurdles at regulated institutions.

Reproducibility

Sell-side desks (Barclays, Morgan Stanley) and buy-side firms with larger books (Lord Abbett, LibreMax) need to run pricing across multiple bonds quickly. The system must support batch or rapid sequential pricing queries with consistent methodology across runs.

2. Functional Requirements

The system must:

- Accept a CLO tranche identifier and set of bond characteristics (coupon, MVOC, manager tier, tenor, etc.) as input
- Return a ranked list of comparable securities with similarity scores and feature-level explanations
- Generate an OAS / pricing estimate via a model with feature importance output
- Support batch pricing queries for multiple tranches in a single session
- Display confidence intervals or uncertainty signals alongside pricing outputs
- Log all inputs, model versions, and outputs for audit and reproducibility
- Potentially, generate a structured investment memo summarizing the comp set, pricing rationale, and key risk factors via LLM. This was not targeted as a major need, but it could be an addition subject to time constraints.

3. Non-Functional Requirements

Performance

Comps retrieval and pricing inference should complete within a few seconds per bond. Batch runs of 10-20 tranches should complete within a single working session without manual intervention.

Reliability

The pricing model must produce consistent outputs given the same inputs; non-determinism in the similarity or pricing layer would undermine practitioner trust. Model versioning must be tracked so that output drift is detectable.

Security

CLO pricing data and trade-level comps are commercially sensitive. The system should ensure that no proprietary client data is used to train shared model versions without explicit consent.

Scalability

New trade color should flow seamlessly into the OAS/pricing model without too much manual intervention. The model must be able to update based on the new trade color that is received, and model results should not deviate too much when given new data otherwise it may undermine practitioner trust.

Explainability

Given that regulated institutions (insurance portfolios, large asset managers) face internal model validation requirements, every pricing output must be accompanied by human-readable reasoning — which features drove the estimate, which comps were most influential, and how confident the model is.

4. Explicit Constraints

Budget

The project is operating under academic capstone constraints with no external funding. We are currently using the past 1 year of trade data parsed from Bloomberg/Valitana via Grant. We are also relying on synthetic data generation to maximize model results. Open-source ML frameworks will be used and potentially API-based LLM access for memo generation. A production-grade commercial data feed complete with all historical trade color is out of scope for v1.

Timeline

The capstone delivery schedule requires a working prototype by the end of the semester. This necessitates a narrow scope: similarity and pricing/OAS regression on ~800 data points (past 1 yr data) and an interpretable feature set applied to BB-rated tranches only.

Team Skills

The team has various backgrounds to complement each other in this project. Grant is using his background in CLO tranche investing to pull/clean data and focus on model development, ensuring that results are sensible and applying necessary fine-tuning. Danni has a strong background in coding, so she is focusing on the major technology decisions (e.g., frontend, backend, database, ML stack, cloud provider). Zheng is focusing on the architecture of the system, including the data flow description, and external dependencies (APIs, services, data sources). Vijeet is focusing on the technical execution plan, ensuring that the team is meeting key milestones (build, test, deploy, validate), analyzing the technical risks, and staying on top of criteria for model success. All team members will be assisting in model development.

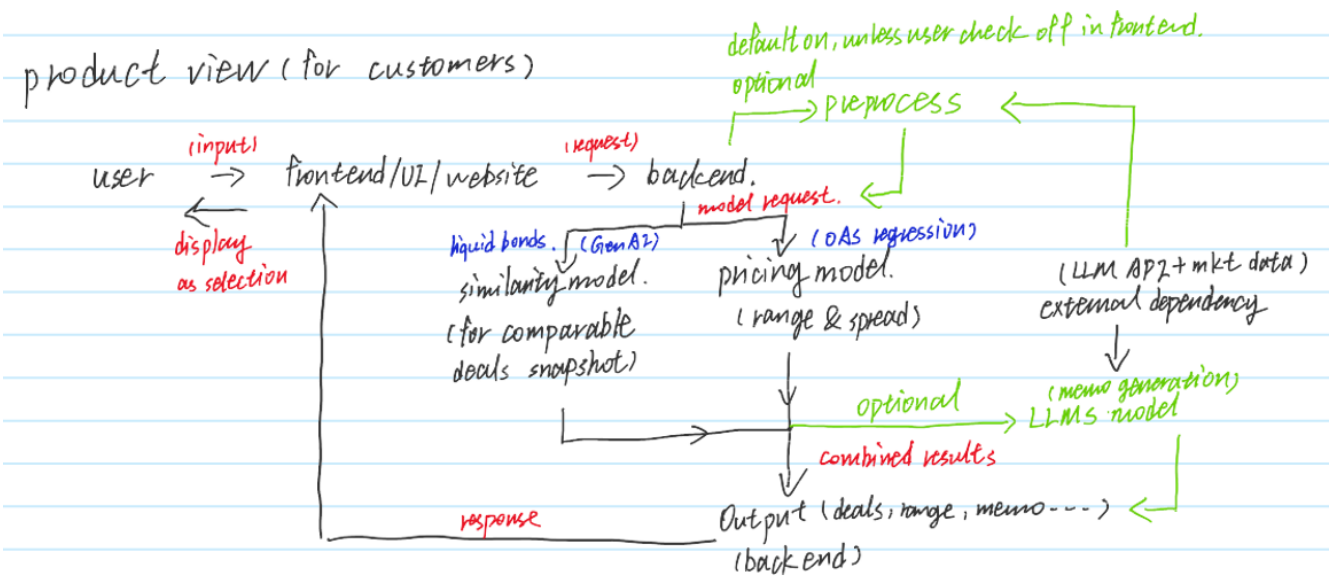
Regulatory & Compliance Considerations

Buy-side institutions — particularly insurance portfolios and large asset managers — are subject to internal model risk management frameworks that require documented validation before any model output can influence investment decisions. The system's explainability requirements (feature importance, comp transparency) are designed to lower this barrier.

CP4 architecture

1. Diagrams

The proposed architecture should consist of two connected layers: a primary training pipeline and a secondary product layer. Here, the training pipeline should be the priority, since the system does not need to be production ready.



For production view or product layer, (as you see on the arrow) the user interacts with the frontend interface to enter the parameter input. Then, the frontend sends data to the backend, and the backend requests the model or calls preprocess module if the data was not cleaned by the user. The results generated by those models will be collected either directly combined in the backend or moved to LLM for memo generation first and combined later. After all, those combined results return to the frontend and presents.

For training view, we preprocess the historical Excel data, like those actions in the parentheses: clean None value, transform the date variable, combine or create new features, etc. Later, a synthetic data generator will get involved to extend the dataset, which will be used to train and test the similarity and pricing model later. After everything looks good, we can add backend and frontend (minimal) to complete the system.

2. Data Flow

Everything will begin with the historical data set (about 793 rows * 38 columns of data, excluding title and blank rows). It contains structural variables (MVOC, attach, thickness, etc.), time variables (trading dates, closing dates, etc.), and valuation, such as spread.

For the training case:

- As mentioned before, we first clean the raw dataset, including handling missing values and inconsistent entries. Detailed preprocessing is then applied to transform the data into valid inputs for specific models.
- Then the GAN or synthetic data generator gets involved to generate data that is almost as true as the original one to extend the dataset.
- And we use the new dataset to train and validate the model. After the model work, implement it in the user case.

For the customer case:

- Inputs from the user are collected through the frontend and sent to the backend. The backend call modules or preprocess data by calling preprocessing module (using a similar preprocessing pipeline) first if not be handled by the user.
- Then the processed features move to the similarity module and pricing module to generate comparable deals and an estimated valuation range.
- Later, it aggregates the result directly, or it first goes through the LLM for memo generation and combines later.
- In the end, the backend sends the result back to the frontend, and the frontend displays it to the user.

3. External dependencies:

- LLM API: used for generating explanations or memos with provided results
- Market data sources: since the Excel column contains VIX, LSTA, or HY CDX, those are real data and should not be generated by the generator.
- Python libraries such as TensorFlow or Pandas: used to support data transformation and model generation.

4. Core System Component (Priority from high to low):

- Preprocessing (both training and product): where we convert raw inputs to feature vectors or a data frame, including data cleaning, encoding, normalizing, etc. Again, the same logic appears in both training and product stages.
- Synthetic Data Generation / GAN (training, not for product, but crucial to the project): where we generate additional samples to extend the dataset.
- Similarity and Pricing Module (both training and product view, same priority): For the similarity model, we collect comparable historical deals based on feature similarity (e.g., KNN or embedding-based methods). For the pricing model, we estimate valuation metrics (spread, price, range, etc.) with a regression-based model.
- Backend (product view): handle requests (routine requests to the similarity or pricing module) and aggregate results.
- Frontend (UI or website, product view): collect inputs like an Excel file, call the backend, and display the result to the user.

5. Supportive Services:

- LLM API (optional): used to generate explanations and memos of model outputs (e.g., short memos summarizing comparable deals and estimated valuation metrics). It does not and should not affect results from previous modules.
- Market Data Sources: mainly used to fill the market variables (e.g., VIX and LSTA index) that may be included in feature construction or module implementation.

Interfaces (insert arrow with OneNote symbols):

For product (online):

- User ↔ Frontend: the user enters inputs and selects functions he/she wants to the frontend, and the frontend displays the output to the user.
- Frontend → Backend: the frontend sends the data and request to the backend
- Backend → Preprocess: raw data from the backend has been processed (e.g., feature extractions or constructions) here for later usage.
- Backend → Preprocess (optional if done outside) → Similarity/Pricing module: the backend sent the process data to the designed model
- Module → LLM (optional) → Backend: the module returns results like price, spread, and deals to the backend for combination or aggregation. Or it first transfers to the LLM module for memo generation and moves back to the backend.
- Backend → Frontend: the backend returns aggregated results to the frontend.

For training (offline):

- Excel dataset → preprocess: where we clean the data and convert it to structured features.
- Preprocess → synthetic data generation → extend dataset: where features are used to generate additional samples
- Extended dataset → train modules → validation: where we use the extended dataset to train and test the model.
- Validation → application/implementation: After the model tests well, we implement it in the backend for the product system.

Boundary:

- Frontend: it is restricted to collecting user inputs and displaying outputs only. It should not perform any preprocessing, model functions, or data generation (memos). Also, it should not be able to modify the inputs entered by the user unless the user change it by himself/herself.
- Backend: it is responsible for handling model requests and aggregating results. Model logics for Pricing, Similarity, or LLM should not be defined here. No change of inputs here.
- Preprocess: it transforms raw inputs into structured feature vectors or data frames only. Again, no model logics should appear here. And this is the only place/component where the raw data has been edited.
- Similarity module: it is restricted to gathering comparable historical deals based on feature similarity. It does not perform pricing estimation or modify input data.
- Pricing module: it is restricted to estimating valuation metrics (e.g., spread and range) using processed features only. It should not be able to see or edit any raw data, and it should not gather comparable deals or generate memos either.

- LLM module: it is restricted to generating explanations or memos only. Also, it is not able to edit the inputs, and should not perform any functions in the similarity or pricing module.
- Training view/pipeline: it is restricted to train and test models. Only those models that pass the test can be implemented in the product pipeline. In reality, the generated data should not be exposed directly to the user, but here we need to use it to test the system later, so this is an exception.

Part 3 Technical Choice Comparative Analysis Table

Architecture Component	What the module must do	Viable Alternatives	Recommended MVP Choice	Python package / tool	Why this is the best MVP choice
Input ingestion	Accept user input from UI or standardized Excel	Manual form only; Excel upload only; support both	Support both form input and Excel upload	streamlit, pandas, openpyxl	Best matches the architecture's frontend requirement to collect inputs and also supports the existing Excel-driven workflow in the CLO market. It is low-cost and easy to demo at this early stage of initializing the product.
Preprocessing	Clean missing values, encode categories, normalize features, convert raw input to model vectors	Manual ad hoc preprocessing; pandas only; pandas + scikit-learn pipeline	pandas + scikit-learn preprocessing pipeline	pandas, numpy, sklearn.impute, sklearn.preprocessing, sklearn.compose, sklearn.pipeline	This aligns directly with the architecture, which identifies preprocessing as a top-priority core component used in both training and product. A shared preprocessing pipeline improves reproducibility and reduces inconsistency.
Synthetic data generation	Extend the small historical Excel dataset for model training	Bootstrapping/resampling; custom GAN in TensorFlow/PyTorch; tabular synthesizer in SDV	SDV tabular synthesizer, starting with GaussianCopula Synthesizer	Sdv python package	The architecture explicitly includes synthetic data generation/GAN as a crucial training-only module. For a semester MVP, SDV is much easier and cheaper than building a custom GAN for this initial MVP demo, while still supporting tabular synthetic data generation. Gaussian Copula is simpler and more stable; CTGAN can be tried later if time allows.

Architecture Component	What the module must do	Viable Alternatives	Recommended MVP Choice	Python package / tool	Why this is the best MVP choice
Similarity module	Return top comparable historical bonds with similarity scores	Rule-based filtering; KNN over engineered features; embedding-based retrieval	KNN or pairwise_distances over preprocessed structured features	sklearn.neighbors.NearestNeighbors or sklearn.metrics.pairwise_distances	This directly matches the architecture's "KNN or embedding-based methods" language and is the simplest interpretable implementation. It aligns with existing practitioner workflow centered on manual comps and supports the trust-building requirement. We also considered an even simplier pairwise_distance just for early phase MVP demo. We can employ more advanced models for future plans.
Pricing module	Predict OAS / price / valuation range	Linear regression / Ridge; Random Forest; XGBoost	Linear/Ridge as baseline + XGBoost as final MVP model	sklearn.linear_model, xgboost	The architecture calls for a regression-based pricing model. Capstone 3 recommends a narrow, interpretable v1, beginning with OAS regression. Linear/Ridge provides a transparent baseline, while XGBoost offers stronger nonlinear performance on tabular data.
Feature importance / explainability	Show why the pricing output makes sense	No explanation; built-in feature importance only; SHAP explanations	Built-in feature importance first, optional SHAP	matplotlib, pandas, shap	Explainability is a stated non-functional requirement. A simple feature importance chart is enough for MVP and supports user trust. SHAP is useful as a future upgrade if time permits.
Uncertainty / confidence signal	Show that prediction is approximate, not absolute	No uncertainty display; prediction interval from residuals; quantile model	Residual-based confidence band / simple uncertainty flag	numpy, pandas, optional sklearn utilities	Section 1 requires confidence intervals or uncertainty signals. A simple residual-based band from validation errors is much easier than a fully probabilistic model and is sufficient for MVP.

Architecture Component	What the module must do	Viable Alternatives	Recommended MVP Choice	Python package / tool	Why this is the best MVP choice
Backend / orchestration	Receive request, call preprocess, similarity, pricing, aggregate results	All-in-one local app; Flask backend; FastAPI backend	Lightweight Python orchestration, optionally split to FastAPI later	python, optional fastapi	The architecture defines backend as request handler and result aggregator, but not the home of model logic. For MVP, a lightweight Python backend is enough and avoids unnecessary deployment complexity.
Frontend / UI User Interface	Display prediction, feature chart, comps table, uncertainty	Flask templates; React frontend; Streamlit	Streamlit	streamlit, pandas, matplotlib/plotly	The system does not need to be production-ready, and Streamlit is the fastest low-cost way to build a usable web UI for demo and class delivery. It supports forms, tables, and charts with minimal code.
Optional LLM explanation	Convert structured results into a short natural-language explanation (lower priority, only if we have time)	direct LLM API prompt; LangChain/agent framework for future improvements	Direct API prompt only	openai Python SDK	The architecture lists LLM API as an external dependency for explanations or memos, but we have deprioritized memo generation for v1. Therefore, a lightweight prompt-based explanation layer is more feasible than agent frameworks or training if we are going to implement this feature. But it is optional for the MVP implementation for now. It is included in the plan.

Part 4. Technical Execution Plan

1. Technical MVP

The MVP is a **working prototype that does the CLO tranche pricing** in a more systematic, quantitative and repeatable way than the current manual process. It will take tranche inputs, preprocess them, retrieve comparable historical deals, and generate an estimated OAS or pricing range with basic interpretability and an uncertainty signal. This matches the earlier requirements around comps identification, pricing, reproducibility, and explainability.

We have secured an industry grade data sample of 750 to 800 records with more than 25 features to start from. Given that we are skewed more towards the features than instances, we will be building more synthetic data from our sample to avoid overfitting.

2. Build Scope

Core build

The core build will include:

- a shared preprocessing pipeline
- synthetic data generation to support model training
- a similarity module for comparable deal retrieval
- a pricing model for OAS or valuation range
- a baseline model for comparison
- feature importance and a simple confidence signal

This is consistent with the technical choices already made, including a preprocessing pipeline, SDV-based synthetic data generation, KNN or pairwise-distance similarity, and Linear or Ridge regression as a baseline with XGBoost as the stronger final model.

Product buffer

If time permits, the team will also package the system into a lightweight UI for demo purposes. If time is tight, the team will present the model through a static prototype interface showing how the final user flow would look. This also fits the architecture, which treats the training pipeline as the priority and the frontend as a later layer.

3. Milestones and Timeline

1. Prepare the date (April 4 to April 10)

The team will finalize the working dataset, clean missing and inconsistent values, standardize variables, and explore feature distributions and correlations. The team will also generate and validate synthetic data, build the first version of the similarity engine, and implement baseline pricing models.

2. Build the first modeling layer (April 11 to April 16)

This stage will include feature reduction work such as filtering, transformation, and PCA testing. At this point, the goal is to have a functioning offline pipeline from cleaned data to comps and estimated pricing.

3. Refine and validate (April 17 to April 21)

The team will improve the pricing model, compare it against baseline approaches, and test consistency, interpretability, and uncertainty outputs. The main focus here is model readiness, not polish.

4. Package and present (April 22 to April 25)

The final days are reserved for packaging results, preparing the demo, and polishing the presentation. If a lightweight UI is feasible, it will be added here. Otherwise, the team will show a static interface prototype and walk through the workflow clearly.

4. Team Responsibilities

- Vijeet will take help of Dannie on synthetic data and support overall integration.
- Dannie and Grant will spend more time on macro data, cleaning, and pricing-model development, with support from Zheng and Vijeet.
- Zheng with Grants support will spend more time on the similarity module and comparable-deal logic.
- Presentation preparation will be coordinated by Vijeet, with all team members contributing to technical explanation, review, and delivery for consistency.

5. Risks and Mitigation

Data risk

The dataset is still small relative to the complexity of the problem, which creates overfitting risk. This will be managed through careful preprocessing, conservative model choices, synthetic data support, and testing on holdout data.

Feature risk

The dataset has many variables, and some may be highly correlated or low signal. Feature screening and PCA testing will be used to simplify the model and improve stability.

Industry-readiness risk

This is not only a coding project. For a pricing tool to be credible in an institutional setting, the output must be interpretable, reproducible, and commercially sensible. A black-box result would be difficult to trust or validate internally. To address this, the system will show comparable deals, feature-level reasoning, and an uncertainty signal alongside the estimate.

Scope risk

There is a risk of spending too much time on frontend or optional features before the core model is stable. The team will therefore prioritize the offline training and validation pipeline first, then use any remaining time for UI and demo polish.

6. Success Criteria

The project will be considered technically ready if it can:

- preprocess tranche inputs consistently
- return commercially reasonable comparable deals
- generate pricing estimates that improve on both a manual comp-style benchmark and a simple regression baseline
- provide feature importance and a confidence signal
- produce stable outputs across repeated runs
- be demonstrated through either a lightweight UI or a clear static interface prototype

This success standard aligns with the earlier requirements around performance, reproducibility, auditability, and explainability.